ACICP302ICS MODULE 1 PRACTICAL LAB

Integrated ICS Network Security & Attacks: Analyze, Test & Recover

Final Practical Lab Report

Learner	JOSEPH AGYAPONG
Environment	Ubuntu 22.04.5 LTS in VirtualBox
Canonical source	labs/ot-security/student-lab-source/
Git commit	2a66f9bcf253eb3fe31c68ad52794cd535421da8
Assessment scope	Loopback-only OT security simulation

1. Executive Summary

This practical assessed the supplied ACICP302ICS OT security simulation against the canonical student-lab-source tree. It first established the simulated architecture and normal baseline, then proceeded to map the Modbus register model, capture and analyze the Modbus/TCP traffic, discover local service exposure, inspect the supplied validation guard, and demonstrate three distinct bounded attacks: false-data injection against the direct plant path, slow pressure drift through the guarded path, and replay of an in-range value through the guarded path.

The findings demonstrated that the supplied design indeed enforced a security boundary by establishing that the direct Modbus service at 127.0.0.1:5020 can be written to freely while the guarded path at 127.0.0.1:5021 used absolute-range, per-write-step, and short-window cumulative-drift checks. Furthermore, the replay attack showcased a particular problem with the supplied guard – writing the same in-range value twice consecutively did not trigger any response from the validation guard. All attacks were demonstrated against the supplied simulator with all traffic directed via loopback. The archive-original source was not deployed, and no bridged networks or port forwarding were established, and the supplied scripts and target parameters were left unmodified.

2. Scope, Safety and Preflight

Canonical source used: labs/ot-security/student-lab-source/.

Git commit verified: 2a66f9bcf253eb3fe31c68ad52794cd535421da8.

Target/bind addresses remained on 127.0.0.1.

No bridged networking, port forwarding, real ICS equipment, or external target was used.

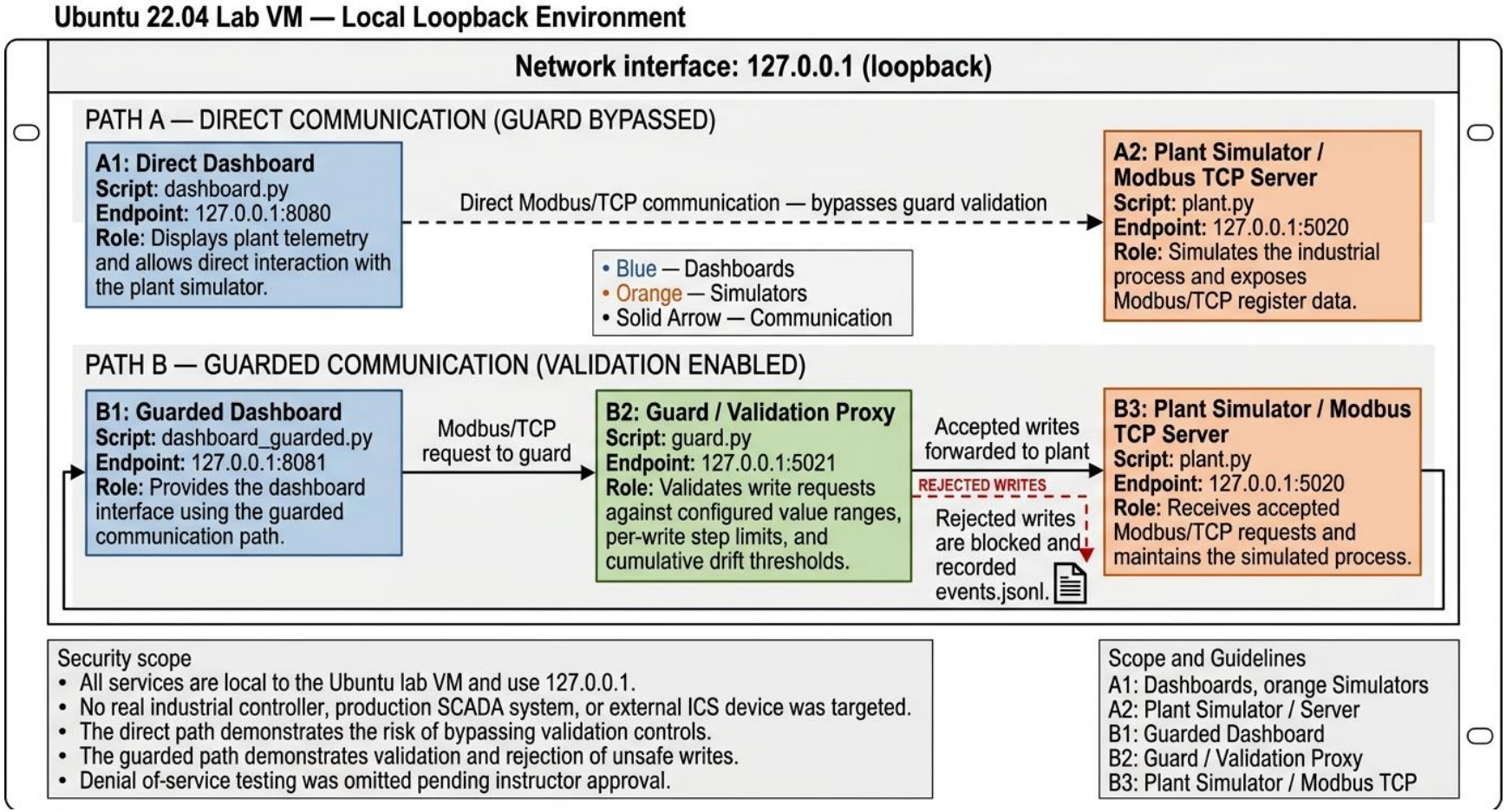
The archive-original directory was not executed.

Temporary NAT was used only for approved setup/download activity and was disconnected before isolated lab execution.

The supplied attack scripts were run with their supplied defaults; no target, bind address, request count, duration, or thread-count changes were made.

The DoS demonstration was not performed because the assessment guide requires instructor approval.

3.Architecture and Data Flow



Use:

Figure 1: OT security lab topology showing direct and guarded Modbus/TCP communication paths within the Ubuntu loopback-only environment.

The supplied reference architecture contains four services:

plant.py — simulated process / Modbus-TCP plant service at 127.0.0.1:5020.

guard.py — validation proxy at 127.0.0.1:5021.

dashboard.py — direct dashboard at 127.0.0.1:8080.

dashboard_guarded.py — guarded dashboard at 127.0.0.1:8081.

Direct path: Dashboard 8080 → Plant 5020.

Guarded path: Dashboard 8081 → Guard 5021 → Plant 5020.

The guard is an educational validation layer. It should not be interpreted as a complete production security boundary or process-safety system.

4. Normal Operation and Data Model

The plant simulation exposes four holding registers to Modbus/TCP clients using function code 3 and unit/slave ID 1.

Register	Signal	Raw range	Engineering range	Meaning
0	Temperature	600–750	60.0–75.0 °C	Simulated temperature
1	Pressure	300–400	30.0–40.0 bar	Simulated pressure
2	Flow	500–1500	50.0–150.0 Nm3/h	Simulated flow
3	Alarm	0/1	0/1	Alarm state

Initial process values are roughly about temperature 670, pressure 340, flow 900. During normal operation, the plant varies temperature by up to ±2 raw units, pressure by up to ±3, and flow up to ±10 for each update. Alarm becomes 1 if pressure reaches or passes raw value of 395.

Normal Dashboard readings were found to be plausible values for the process, and normal operation of the plant can be seen occurring on both the direct and the guarded paths.

5. Packet Visibility and Modbus Analysis

Packet Analysis Limitation

The captured Modbus/TCP traffic was used to identify the request and correlate it with the corresponding response using the transaction identifier and protocol fields. However, the response decoding did not provide sufficiently reliable evidence to independently confirm the returned register value. Therefore, register values and process-state conclusions are interpreted alongside the documented register map and dashboard evidence rather than inferred from an ambiguous packet decode.

A baseline Wireshark capture was taken on the loopback interface and saved as evidence/pcap/base_modbus_capture.pcapng for later analysis. By filtering tcp.port == 5020 and Modbus traffic, it was possible to find out the corresponding request/response transactions.

In this case, transaction 2397 was used as an example. It can be seen that the request and response have the same transaction identifier (2397), protocol identifier (0), and unit identifier (1). The request's length is 11, while the response's length is 6. The Modbus function code 3 is used for reading holding registers.

This packet capture proves the protocol communication and request/response exchange between the client and server. However, it cannot be used as evidence that the process is safe or that the request/response has been authorized.

6. Local Service Exposure

The supplied simulation exposes local TCP listeners for the plant, guard, direct dashboard, and guarded dashboard. The relevant endpoints are 127.0.0.1:5020, 127.0.0.1:5021, 127.0.0.1:8080, and 127.0.0.1:8081.

The main security observation is that the direct plant Modbus endpoint is a separate path from the guard. A client that can reach the direct endpoint can bypass the validation proxy.

7. Guard Analysis

The supplied guard applies three principal validation classes:

Absolute range checks: temperature 600–750, pressure 300–400, and flow 500–1500.

Per-write step limits: temperature 30 raw units (3.0 °C), pressure 50 raw units (5.0 bar), and flow 100 raw units (10.0 Nm3/h).

Three-sample cumulative drift checks: temperature threshold 20 raw units (2.0 °C), pressure threshold 20 raw units (2.0 bar), and flow threshold 40 raw units (4.0 Nm3/h).

Rejected writes are not forwarded to the plant. Accepted writes are forwarded and history is updated. Rejection events are written to events. jsonl.

The guard therefore provides useful behavioral validation, but its protection depends on traffic using the guarded path and on its static thresholds being appropriate for the process.

7.1 Pre-Test Plan

All tests were conducted within the supplied Ubuntu OT simulation using loopback-only endpoints. Normal plant telemetry was checked before testing. Evidence was collected from the attack terminal, dashboards, guard logs, and packet capture where applicable.

Test	Script and target	Baseline	Expected outcome	Evidence	Stop Condition	Recovery
False-data injection	plant_sim/false_data_attack.py; 127.0.0.1:5020	Normal plant telemetry	Unsafe values reach the plant through the unguarded path	04A, 04B	Stop after planned iterations or if the simulation becomes unstable	Stop the attack and confirm normal plant telemetry
Slow pressure drift	plant_sim/slow_drift_attack.py; 127.0.0.1:5021	Normal pressure telemetry	Out-of-range or excessive cumulative pressure changes are rejected	05A, 05B, 14	Stop after planned attempts or if the simulation becomes unstable	Stop the test and confirm normal plant telemetry
Replay attack	plant_sim/replay_attack.py; 127.0.0.1:5021	Normal pressure telemetry and existing guard-event log	Check whether repeated identical pressure writes are	06A, 06B	Stop after planned replay attempts or if the simulation	Stop the test and confirm normal plant telemetry

			detected or rejected		becomes unstable	

All tests were restricted to the local loopback simulation. Testing was stopped after the planned attempts, and recovery was verified by checking that normal plant telemetry was restored. The replay test also documented a detection limitation because repeated identical pressure writes did not generate new guard events.

8. Controlled Validation Tests

Test	Path	Expected	Observed	Evidence	Recovery
False-data injection	127.0.0.1:5020	Unsafe values reach plant	110 °C, 55 bar, 200 Nm3/h reached plant; 6/6 unsafe readings	04A, 04B	Normal direct telemetry restored after attack
Slow pressure drift	Guarded 5021	Out-of-range/drift writes rejected	Writes above 40 bar were rejected; cumulative	05A, 05B, 14	Normal telemetry confirmed

			drift events recorded		
Replay	Guarded 5021	Repeated value ideally detected	Three identical 39.4 bar writes produced no new guard event	06A, 06B	Normal telemetry confirmed

9. Validation Test 1 — False Data Injection

The provided false_data_attack.py was run against port 5020. It cycled through temperature of 110 degrees C, pressure of 55 bar, and flow of 200Nm3/h six times.

Observation: the direct plant accepted the values, although the terminal warned that during all six cycles there were unsafe values of temperature, pressure, and flow and that the unsafe values got through to the plant. The direct dashboard showed the condition that was input. Thus, the direct plant endpoint does not provide protection against the attack.

However, when the demonstration was shut down, the simulated plant returned to normal telemetry, and the dashboard showed normal values.

10. Validation Test 2 — Slow Drift Through Guard

The supplied slow_drift_attack.py was run against port 5021. The script adds 10 raw units to the pressure for 12 iterations.

The observed result is that the guard accepted the values as long as they are within its checks, but it rejected out-of-range and cumulative-drift conditions. Events showed that pressure values such as 40.6, 41.6, 42.6, and 43.6 bar were outside the 30-40 bar safe range, and cumulative drift exceeded the 2.0 bar threshold in three-sample averages in later attempts.

Therefore, the guard can prevent the supplied slow-drift scenario from entering the plant, if the guarded path is taken.

11. Validation Test 3 — Replay Limitation

The supplied replay_attack.py was run against port 5021. It read off the current pressure, chose an in-range value of approximately 39.4 bar, and replayed the value three times.

What happened next: the replay did not trigger a new guard event. Notably, the event log was untouched by the replay. It is a concrete limitation of the supplied guard that repeated in-range writes are not individually identified as replay attacks. The result does not show that every replay technique would evade the guard; it only shows that this particular repeated-value scenario was not detected.

12. Risk Register

ID	Risk	Impact	Assessment	Recommended control
R1	Direct Modbus path bypasses guard	High	Demonstrated	Remove unnecessary direct access; enforce authenticated/authorized access and network segmentation.
R2	No demonstrated client authentication/authorization	High	Observed from supplied design	Use appropriate ICS access control, segmentation and authenticated management paths.

R3	Replay of identical in-range values not detected	Medium	Demonstrated limitation	Add sequence/timestamp/context validation and anomaly detection appropriate to the process.
R4	Static thresholds may not represent all process states	Medium	Design limitation	Tune thresholds from validated process requirements and add contextual rules.
R5	Denial-of-service behavior not assessed	Medium	Not performed; approval required	Perform only under approved test conditions with recovery criteria.
R6	Guard has limited process context	Medium	Design limitation	Combine protocol validation with process-aware monitoring and independent safety controls.

13. Recovery and Evidence Handling

After each controlled demonstration, the simulator was allowed to return to normal telemetry and the result was checked through the dashboard/telemetry. The baseline PCAP was copied into evidence/pcap/. The evidence directory structure was created as:

evidence/screenshots/

evidence/pcap/

evidence/report/

The PCAP was verified at approximately 1.2 MB as evidence/pcap/base_modbus_capture.pcapng.

14. Evidence Index

Evidence	Description	Use
01	Wireshark baseline capture	Baseline protocol visibility
02	Normal direct dashboard	Normal operation
03	Normal guarded dashboard	Normal guarded operation
04A	False-data attack terminal	Direct-path validation
04B	Direct dashboard impact	Unsafe-value impact
Recovery 1	Direct dashboard normal after false data	Recovery
05A	Slow-drift attack terminal	Guarded-path validation
05B	Initial slow-drift rejection output	Guard behavior
06A	Replay attack terminal	Replay validation
06B	No new event after replay	Guard limitation
Recovery 3	Normal plant telemetry after replay	Recovery
07	Local Modbus service exposure	Exposure mapping
08	All four local listeners	Architecture/exposure

09	Matching Modbus transaction 2397	Packet analysis
10	Register map/ranges	Data model
11	Direct dashboard baseline	Normal baseline
12	Guarded dashboard baseline	Normal baseline
13	Guard source checks/thresholds	Guard analysis
14	Guard rejection events	Validation evidence

Recovery Verification

Recovery was verified by checking the plant telemetry after the attack tests. The direct dashboard and normal plant telemetry evidence showed that the simulation had returned to normal operating values. Recovery checks were performed after the false-data injection and slow pressure drift tests. The replay test also ended with normal telemetry confirmed. These checks demonstrate recovery of the local simulation; they do not establish recovery of a live industrial control system**.**

Recovery Evidence

- **Recovery-01-Direct-Dashboard-Normal.png —** evidence of normal readings on the direct dashboard.
- **Recovery-03-Normal-Plant-Telemetry.png —** evidence of normal plant telemetry after testing.

Both screenshots are stored in evidence/screenshots/ in the evidence repository.

15. Evidence-to-Rubric Mapping

Rubric area	Evidence / sections	Coverage
1. Source, scope and architecture	Sections 2–3; commit; Evidence 07–08	Canonical source, loopback scope and architecture established.
2. Data model and normal operation	Section 4; Evidence 02–03, 10–12	Register map, engineering ranges and baseline behavior documented.
3. Packet visibility and exposure	Section 5–6; Evidence 01, 07–09	Modbus/TCP traffic and local service exposure analyzed.
4. Guard analysis	Section 7; Evidence 13–14	Range, step, drift logic and rejection events documented.
5. Validation and recovery	Sections 8–13; Evidence 04–06 and recovery evidence	Three bounded scenarios, observed results and recovery documented.
6. Report and defence	This report; risk register; evidence index	Findings, limitations, risks and traceability consolidated.

16. Source Traceability

Repository: ACICP302ICS-Network-Security-and-Attacks.

Canonical lab path: labs/ot-security/student-lab-source/.

Verified commit: **2a66f9bcf253eb3fe31c68ad52794cd535421da8.**

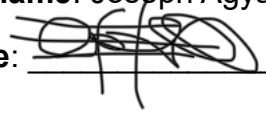
Relevant source files reviewed: plant.py, guard.py, dashboard.py, dashboard_guarded.py, false_data_attack.py, slow_drift_attack.py, replay_attack.py and supporting documentation.

No source modification was required for the demonstrations documented in this report.

17. Safety Declaration

I confirm that this practical was performed only against supplied educational OT simulation in assigned Ubuntu VM. I did not target real industrial control equipment, external networks, campus/home systems or another VM. I have kept loopback-only operation, have not used bridged networking or port forwarding, have not executed the archive-original source and have not modified supplied attack target parameters. I did not perform DoS demonstration since it required instructor's approval.

Learner name: Joseph Agyapong

Signature: 

Date: 7th October 2026

18. Conclusion

Informed by the practical, the given simulation can be considered an effective proof of concept in terms of understanding the ICS protocol visibility, direct/guarded paths, and the basics of process-value validation. What stands out as the most informative result of the practical is that the direct Modbus service can indeed be written to, without being subjected to the guard in place. At the same time, the guarded path was successfully able to block the slow drift attempts that were used in the testing. The replay demonstration was also insightful, highlighting one significant detection gap in the current approach – the repeated in-range writes that were attempted would not trigger a violation due to being identical.

Overall, the simulation was an exercise in showcasing the theoretical benefits of such a control mechanism. In practice, it would have to be supplemented with additional protective measures, such as network segmentation, restricted access

control rules, monitoring, process-aware detection mechanisms, safety systems, and carefully approved testing procedures.

19. Submission Checklist

- Final report completed.
- PCAP copied to evidence/pcap/base_modbus_capture.pcapng.
- Evidence subdirectories created.
- Screenshots should be placed in evidence/screenshots/ using the Evidence Index names where possible.
- Before final submission, stop the four lab services and verify that ports 5020, 5021, 8080 and 8081 are no longer listening.
- Keep the learner evidence repository private if required by the assessment brief.
- Final report should be placed in FINAL_REPORT/.